

Introduzione ai sistemi Distribuiti

Terminologia e modelli

Roberto Marino, PhD

Università di Messina
CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

14 maggio 2026

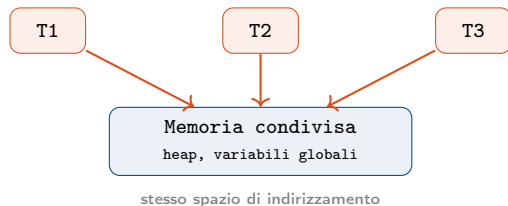


UniMe
1548

La programmazione concorrente — il modello che conosci

Nella programmazione concorrente il contesto è fisso e garantito dall'hardware e dall'OS. Non serve dichiararlo: è sempre lo stesso.

- ▶ n thread sullo **stesso processo**, nello stesso spazio di indirizzamento
- ▶ **Memoria condivisa**: ogni thread vede l'heap degli altri
- ▶ **Clock unico**, fornito dall'OS: tutti i thread condividono la stessa nozione di tempo
- ▶ Coordinamento tramite **mutex**, **semaphore**, **monitor** — primitive che assumono la memoria condivisa come substrato
- ▶ Se un thread muore, il processo se ne accorge immediatamente



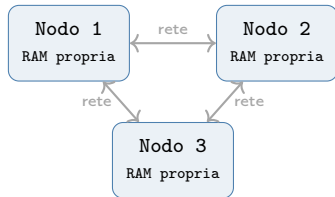
Garanzia implicita: se ho il lock, nessun altro può modificare la struttura dati protetta. Questo è garantito dalla macchina, non dal codice.

Si elimina la memoria condivisa: cosa non funziona più?

In un sistema distribuito i nodi sono macchine separate. Non condividono nulla: né RAM, né clock, né OS.

Ogni primitiva della programmazione concorrente che *assume* la memoria condivisa **smette di avere senso**:

- ▶ Il **mutex** non esiste: non c'è memoria su cui fare lock
- ▶ Il clock condiviso non esiste: ogni nodo ha il proprio orologio fisico, che deriva nel tempo rispetto agli altri
- ▶ La morte di un nodo non è osservabile direttamente: nessun segnale, nessuna eccezione
- ▶ Non abbiamo più a disposizione il concetto di atomicità



La rete introduce ritardi, perdite e riordini.
Niente di tutto questo esiste nella comunicazione tra thread sulla stessa macchina.

Il messaggio come unico strumento di coordinamento

Senza memoria condivisa, l'unico modo per coordinare due nodi è **scambiarsi messaggi**.

Questo non è solo un vincolo implementativo: cambia il modo in cui si ragiona sulla correttezza.

Cosa sa un nodo in ogni istante

- ▶ Il proprio stato locale
- ▶ I messaggi che ha ricevuto fino a quel momento
- ▶ Niente altro è certo

Ogni conoscenza sul resto del sistema è *indiretta*: arriva via messaggio, quindi può essere **in ritardo**, **persa** o **falsa**.

Le domande che ogni algoritmo deve rispondere

1. Come si coordinano i nodi se i messaggi possono perdersi?
2. Come si rileva che un nodo è crashato se non arriva nessun segnale?
3. Come si mantiene la coerenza se due nodi voglio modificare lo stesso record?
4. Come si garantisce il progresso se non si sa quando arriverà il prossimo messaggio?

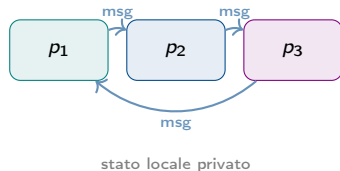
Definizione

$$\Pi = \{p_1, p_2, \dots, p_n\}$$

Insieme finito e noto di processi *sequenziali*.

- ▶ Ogni p_i ha **stato locale** proprio, non accessibile dagli altri
- ▶ I processi comunicano **solo** via messaggi
- ▶ Il parametro f (maximum failing nodes): numero massimo di processi difettosi che l'algoritmo tollera
- ▶ Un processo è **corretto** in un'esecuzione se non subisce guasti in quell'esecuzione

f è una *garanzia dell'algoritmo*, non una proprietà della rete: se i guasti superano f , l'algoritmo non offre più nessuna garanzia.



Confronto con la concorrenza: i thread condividono l'heap; due processi distribuiti non condividono assolutamente nulla. Ogni dato che uno dei due conosce dell'altro è arrivato via messaggio.

Esecuzione locale sequenziale

All'interno di p_i le operazioni avvengono in ordine, una alla volta. Non esistono race condition interne. Questo semplifica il ragionamento: lo stato locale è sempre ben definito.

Progresso garantito

Un processo corretto che non è in attesa di un messaggio continua ad eseguire. Non si blocca per indecisione interna.

Cosa non assumiamo

- ▶ Velocità relativa tra processi: p_1 può essere ordini di grandezza più lento di p_2
- ▶ Ordine di arrivo dei messaggi — salvo garanzie esplicite del canale
- ▶ Conoscenza globale: ogni processo vede solo il proprio stato e i messaggi ricevuti

Conoscenza comune iniziale

I processi conoscono n e f . Ogni altra informazione condivisa deve essere comunicata esplicitamente via messaggio. Non esistono variabili globali condivise per definizione.

Processo corretto

Un processo è **corretto** in una specifica esecuzione se non subisce guasti durante quell'esecuzione.

È una proprietà relativa all'esecuzione, non al processo in astratto: p_i può essere corretto in un'esecuzione e crashare in un'altra.

Algoritmo corretto

Un algoritmo è **corretto** rispetto a un modello se arriva al risultato, risolvendo il problema, in *tutte* le esecuzioni ammesse dal modello, incluse quelle peggiori che l'avversario può costruire (nonostante i crash).

Le proprietà dell'algoritmo sono definite in termini di processi corretti:

- ▶ *Agreement*: tutti i **processi corretti** decidono lo stesso valore
- ▶ *Termination*: ogni **processo corretto** prima o poi decide

Ora che sappiamo cosa sono i processi, possiamo definire cosa intendiamo per **modello** di sistema distribuito.

Il modello di un algoritmo distribuito è l'insieme delle **ipotesi sul sistema** che l'algoritmo dà per vere senza doverle verificare a runtime. Si compone di tre assi:

- ▶ **Guasto:** cosa può andare storto con i processi?
- ▶ **Tempo:** quanto è affidabile il clock e la consegna dei messaggi?
- ▶ **Canale:** i messaggi si perdono? possono essere alterati?

Perché dichiararlo esplicitamente

In concorrenza il modello è *implicito*: garantito da hardware e OS, uguale per tutti. Puoi ignorarlo — raramente è catastrofico.

Nei sistemi distribuiti il modello è una *scelta di design*: infrastrutture diverse offrono garanzie diverse. Un algoritmo è corretto **solo rispetto al modello per cui è stato progettato**.

Cambia il modello, cambia la correttezza.

Nelle prossime sezioni costruiamo i tre assi uno per uno.

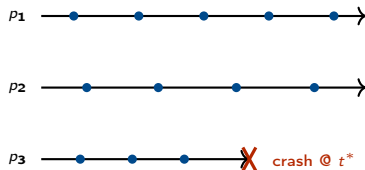
Definizione

Il processo esegue correttamente fino a un istante t^* , poi **si ferma per sempre**:

- ▶ Non manda mai messaggi errati
- ▶ Non si risveglia dopo il crash
- ▶ Non emette nessun segnale: gli altri non sanno *quando* è crashato, né se è crashato o solo lento

Analogia: come un thread che riceve SIGSEGV. Smette di esistere senza corrompere gli altri. Ma in distribuito nessuno riceve il segnale.

È il modello più semplice e il più usato: Paxos, Raft, Viewstamped Replication assumono tutti crash-stop.



Limite del modello

Non copre i sistemi in cui i nodi si riavviano (database, server di produzione). Per quelli serve crash-recovery.

Definizione

Il processo può **crashare e ripartire** un numero arbitrario di volte.

La distinzione chiave è tra due tipi di stato:

- ▶ Stato **volatile** (RAM): azzerato al crash
- ▶ Stato **persistente** (disco): sopravvive

Gli algoritmi devono fare *checkpoint* prima di ogni operazione critica per garantire che, al riavvio, il processo sappia cosa aveva promesso.

Il problema del risveglio

Al riavvio il processo non sa cosa aveva promesso prima del crash — a meno di averlo scritto su disco. Un voto dimenticato può rompere l'invariante del quorum.



Esempi reali: PostgreSQL WAL, Kafka log, ZooKeeper snapshot + transaction log. Ogni sistema che vuole durabilità usa questo modello.

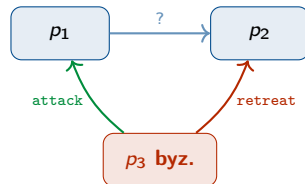
Nei modelli crash-stop e crash-recovery un processo difettoso si *tace*: smette di inviare messaggi o li invia come avrebbe sempre fatto. Non causa mai danni *attivi*.

Il modello **Byzantine** rimuove questa ipotesi: un processo difettoso può fare **qualsiasi cosa**.

Il problema dei generali (Lamport 1982)

n generali devono concordare, cioè devono raggiungere un **consenso**: attaccare o ritirarsi. Alcuni sono traditori e inviano messaggi contraddittori a destinatari diversi per impedire l'accordo.

La domanda è: quanti traditori può tollerare il sistema e raggiungere ancora il consenso?



p_3 invia messaggi diversi a p_1 e p_2 . Senza sapere se p_3 è un traditore, p_1 e p_2 non possono accordarsi in modo sicuro.

Cosa può fare un processo Byzantine

- ▶ Inviare valori arbitrariamente sbagliati
- ▶ Inviare messaggi *diversi* a destinatari diversi nella stessa fase del protocollo
- ▶ Non rispondere strategicamente
- ▶ Impersonare altri processi (se non ci sono firme crittografiche)
- ▶ Colludere con altri processi Byzantine

Risultato formale

Per tollerare f processi Byzantine su n totali:

$$n \geq 3f + 1$$

Con $n = 3f$ il consenso è **impossibile**. Crash-stop richiede solo $n \geq 2f + 1$.

Quando è necessario

- ▶ Nodi **compromessi** da un attaccante
- ▶ Hardware con bit-flip in ambienti critici (avionica, spazio, medicina)
- ▶ Sistemi **senza fiducia** reciproca tra partecipanti (blockchain)

In concorrenza tutti i thread leggono lo stesso clock hardware. L'OS garantisce che `clock_gettime()` restituisca un valore coerente per tutti. Il tempo è un substrato condiviso, come la RAM.

Nei sistemi distribuiti ogni nodo ha il proprio oscillatore fisico. I clock **non sono sincronizzati per default**:

- ▶ Il clock drift tipico è ~ 100 ppm, pari a circa 8 secondi al giorno
- ▶ NTP riduce lo skew a qualche millisecondo, ma non lo azzerà
- ▶ Una GC pause può bloccare un nodo per centinaia di millisecondi
- ▶ Un timeout può scadere mentre il nodo è solo **lento**, non morto

Definizione

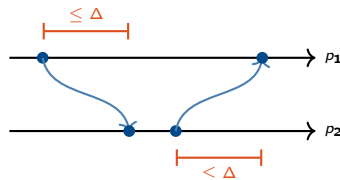
Esiste un bound Δ **noto e finito** tale che:

- ▶ ogni messaggio arriva entro Δ
- ▶ ogni passo di calcolo dura al più Δ

Con Δ noto, un timeout è affidabile: se non risponde entro Δ , il processo è certamente crashato. Si ottiene un **failure detector perfetto**.

Il consenso è risolvibile in modo deterministico, il ragionamento sugli algoritmi è molto più semplice, ed è possibile costruire algoritmi ottimali per round.

Limite: irrealistico su reti general purpose. GC pause, congestione TCP e scheduling del kernel violano Δ in modo imprevedibile.



Usato in sistemi embedded real-time, reti industriali (PLC, CAN bus, TT-Ethernet), avionica.

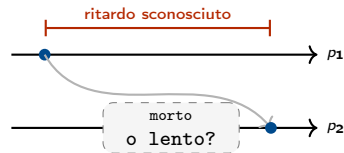
Definizione

Nessun bound temporale: i messaggi arrivano prima o poi, ma non si sa quando. I passi di calcolo possono richiedere tempo arbitrario.

- ▶ Modello più fedele a Internet reale
- ▶ **Impossibile** distinguere nodo morto da nodo lento: non esiste timeout affidabile
- ▶ **Impossibile** costruire un failure detector con garanzie forti

Perché studiarlo

Il caso asincrono è il **pavimento teorico**: ogni risultato di impossibilità qui vale universalmente. Un algoritmo che funziona in asincrono funziona in qualsiasi modello più forte.



Il dilemma

p_1 attende risposta da p_2 . Aspettare ancora, o dichiararlo crashato? In asincrono non esiste risposta corretta e deterministica. Questo porta direttamente al teorema FLP.

Il teorema FLP — il problema fondamentale

In un sistema asincrono un processo non riceve risposta da un altro. Cosa fa?

Ha due opzioni:

- ▶ **Aspetta ancora** — ma se il processo è crashato, aspetterà per sempre
- ▶ **Decide senza di lui** — ma se il processo è solo lento, potrebbe decidere un valore diverso da quello che deciderà lui quando si sveglierà

In asincrono non esiste modo di distinguere i due casi. Nessun timeout è affidabile. Nessuna attesa è sufficientemente lunga da garantire che il processo sia davvero morto.

Fischer, Lynch e Paterson (1985) dimostrano che questa ambiguità non è aggirabile: non esiste nessun algoritmo deterministico che la risolva, anche con un solo crash possibile.

Enunciato

In un sistema **completamente asincrono**, in cui anche **un solo** processo può crashare, nessun algoritmo deterministico può garantire simultaneamente:

- ▶ **Agreement** — tutti i processi corretti decidono lo stesso valore
- ▶ **Termination** — ogni processo corretto prima o poi decide

Le ipotesi sono le più favorevoli possibili: un solo crash, crash-stop, niente Byzantine. Nonostante questo, il consenso è impossibile.

Non è un limite ingegneristico. È una dimostrazione matematica: non si può aggirare restando nel modello.

Cosa non dice FLP

Non dice che il consenso è impossibile in pratica. Dice che serve uscire dal modello asincrono puro — aggiungere qualcosa che rompa l'ambiguità tra nodo lento e nodo morto.

Ogni sistema reale che risolve il consenso ha scelto una di tre uscite:

Partial synchrony — si assume che la rete prima o poi si stabilizzi. La termination è garantita solo dopo la stabilizzazione. Paxos e Raft usano questa strada.

Randomizzazione — un processo che non sa cosa fare lancia una moneta. La termination è garantita con probabilità 1, non con certezza.

Failure detector — un modulo esterno decide chi è morto e chi no, delegando fuori dall'algoritmo il problema che FLP rende impossibile.

Fair-loss

- ▶ I messaggi possono **perdersi**
- ▶ Ritrasmettendo abbastanza a lungo, il messaggio **prima o poi arriva**
- ▶ Nessuna garanzia sull'ordine
- ▶ Il livello fisico reale: UDP

Costruire sopra:
ritrasmissioni + numeri di sequenza

Reliable

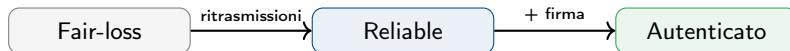
- ▶ **Nessuna perdita** se mittente e destinatario sono corretti
- ▶ Si costruisce sopra fair-loss
- ▶ Approssimato da TCP
- ▶ Sufficiente per crash-stop

Costruire sopra:
causal broadcast, total order

Autenticato

- ▶ Ogni messaggio porta una **firma crittografica**
- ▶ Necessario contro Byzantine: senza firma un nodo malevolo può impersonare gli altri
- ▶ Costo: operazioni crypto a ogni messaggio

Usato da:
PBFT, HotStuff, protocolli BFT



Safety — “il sistema non farà mai una certa cosa”

Immagina un bancomat. La regola è: non erogare soldi se il saldo non è sufficiente.

La safety è l'impegno del sistema a rispettare questa regola **in ogni momento**, senza eccezioni. Non importa il carico, non importa se la rete è lenta, non importa se un nodo è appena tornato online. In qualsiasi istante tu guardi il sistema, quella regola è rispettata.

Definizione

Una proprietà è di **safety** se è un impegno su *cosa il sistema non farà mai*. Si verifica istante per istante: in qualsiasi momento si osservi il sistema, la proprietà è soddisfatta.

Esempi nei sistemi distribuiti

- ▶ Due nodi non decidono mai valori diversi
- ▶ Un messaggio non viene mai consegnato due volte
- ▶ Due processi non entrano mai contemporaneamente in sezione critica

Implicazione per il design

La safety non ha scuse e non dipende dal tempo o dalla rete. Deve valere sempre — durante una partizione, sotto carico, pre-GST.

È la proprietà che un sistema non può mai sacrificare: un sistema che viola la safety ha fatto qualcosa di sbagliato e irreversibile. I soldi sono già usciti dal bancomat.

Liveness — “il sistema farà prima o poi una certa cosa”

Stesso bancomat. Una seconda regola: se il saldo è sufficiente, il prelievo deve prima o poi andare a buon fine.

La liveness è l'impegno del sistema a fare progresso: non basta non fare cose sbagliate — il sistema deve anche fare le cose giuste, prima o poi.

Definizione

Una proprietà è di **liveness** se è un impegno su *cosa il sistema farà prima o poi*. Non si verifica in un singolo istante: si verifica sull'intera esecuzione, chiedendo che qualcosa di buono accada in un tempo finito.

Esempi nei sistemi distribuiti

- ▶ Ogni processo corretto prima o poi decide
- ▶ Ogni messaggio inviato viene prima o poi consegnato
- ▶ Il sistema prima o poi elegge un leader

Perché è più difficile da garantire

La liveness richiede che il sistema faccia progresso, e il progresso dipende dal tempo. In un sistema asincrono, dove i messaggi possono ritardare arbitrariamente, non si può garantire che qualcosa accada entro un tempo finito.

Questo è esattamente ciò che dimostra FLP: in asincrono puro la liveness del consenso non si può garantire, perché non si può escludere che il sistema rimanga bloccato per sempre senza fare nulla di sbagliato.

Safety vs Liveness — il compromesso fondamentale

Safety e liveness tirano in direzioni opposte.

Un sistema che **non fa mai nulla** soddisfa tutta la safety del mondo: non decide mai valori sbagliati, non consegna mai messaggi duplicati, non viola mai la mutua esclusione. Ma viola ogni liveness: non fa mai progresso.

Un sistema che **risponde sempre subito** soddisfa tutta la liveness del mondo: ogni richiesta ottiene una risposta immediata. Ma rischia di violare la safety: risponde senza coordinarsi con gli altri, e due nodi possono rispondere cose diverse.

Il design di ogni algoritmo distribuito implica la scelta di "dove stare" tra questi due estremi.

La scelta durante una partizione di rete

I nodi A e B non riescono a comunicare. Arriva una richiesta su A.

- ▶ **A si blocca** in attesa di B $\Rightarrow$ **safety** preservata, **liveness** violata
- ▶ **A risponde** senza aspettare B $\Rightarrow$ **liveness** preservata, **safety** a rischio

Questa non è una domanda tecnica. È una domanda sul sistema: cosa è più pericoloso — rispondere lentamente o rispondere in modo potenzialmente inconsistente?

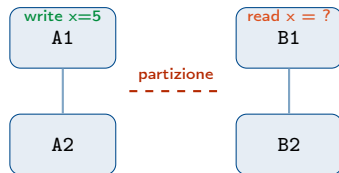
Il teorema CAP

Immaginiamo un database replicato su più nodi. I dati sono copiati su ognuno per resistere ai guasti. Gli utenti leggono e scrivono su nodi diversi.

Tutto funziona finché i nodi comunicano. Il problema emerge quando la rete si **partiziona**: un gruppo di nodi non riesce a raggiungere l'altro. Non è un crash — i nodi sono vivi, ma isolati.

In quel momento arriva una scrittura su un nodo del gruppo A e subito dopo una lettura sullo stesso dato da un nodo del gruppo B.

B non sa che A ha scritto qualcosa di nuovo. Cosa risponde?



B può rispondere 5 solo se riesce a raggiungere A. Ma c'è la partizione. Risponde il vecchio valore, o non risponde?

Brewer (2000) congettura, Gilbert e Lynch (2002) dimostrano: un sistema distribuito non può garantire simultaneamente queste tre proprietà.

Consistency

Ogni lettura riceve il valore più recente scritto, oppure un errore.

Availability

Ogni richiesta riceve sempre una risposta.

Partition tolerance

Il sistema continua a funzionare anche se la rete si partiziona e alcuni nodi non riescono a comunicare con altri.

Perché non si possono avere tutte e tre

La partition tolerance non è negoziabile: le reti reali si partizionano, non si può semplicemente ignorarlo.

Durante una partizione, ogni sistema deve scegliere:

- ▶ **CP** — consistency + partition tolerance. Si blocca fino a che la partizione si risolve. Risponde con un errore piuttosto che con un dato vecchio. Esempi: HBase, Zookeeper, etcd.
- ▶ **AP** — availability + partition tolerance. Risponde sempre, anche con un dato potenzialmente non aggiornato. Esempi: Cassandra, CouchDB, DynamoDB.

Il teorema CAP — come usarlo e i suoi limiti

Come usarlo

CAP è uno strumento di ragionamento, non una legge di fisica. Serve a fare una domanda precisa al momento del design: durante una partizione, cosa è più pericoloso per il mio sistema — bloccarsi o rispondere con dati vecchi?

Un sistema bancario non può permettersi dati inconsistenti: sceglie CP. Un sistema di raccomandazioni può permettersi di mostrare contenuti leggermente vecchi: sceglie AP.

Il limite principale

CAP tratta consistency e availability come scelte binarie: o le hai o non le hai. La realtà è uno spettro continuo.

I livelli di consistency

Dire “sacrifico consistency” nasconde un mondo di sfumature:

- ▶ *Strong consistency* — tutti i nodi vedono sempre lo stesso valore
- ▶ *Causal consistency* — chi ha visto una scrittura vede anche tutto ciò che l’ha causata
- ▶ *Eventual consistency* — i nodi convergeranno sullo stesso valore, ma non si sa quando

Comunicazione

- ▶ **Reliable broadcast** — un messaggio inviato da un nodo corretto viene consegnato a tutti, anche se il mittente crasha a metà. *Eager reliable broadcast*.
- ▶ **Causal broadcast** — se A ha causato B, tutti i nodi vedono A prima di B. *Vector clock broadcast*.
- ▶ **Total order broadcast** — tutti i nodi consegnano i messaggi nello stesso ordine. Equivalente al consenso. *Paxos, Raft*.

Coordinamento

- ▶ **Elezione del leader** — i nodi devono accordarsi su quale di loro coordina le operazioni del cluster. *Bully, Ring election, Raft*.
- ▶ **Mutua esclusione** — una risorsa condivisa viene usata da un solo nodo alla volta, senza un lock centrale. *Ricart-Agrawala, token ring*.
- ▶ **Barriera di sincronizzazione** — tutti i nodi aspettano che tutti abbiano raggiunto un punto prima di procedere. *Barriera a due fasi*.

Dati

- ▶ **Atomic commitment** — una transazione distribuita viene applicata su tutti i nodi o su nessuno, anche se il coordinatore crasha. *Two-phase commit, three-phase commit.*
- ▶ **Replicazione** — copie coerenti di un dato su più nodi, con garanzie di consistenza scelte esplicitamente. *Paxos, Raft, Viewstamped Replication.*
- ▶ **Sharding** — i dati vengono distribuiti su nodi diversi e ribilanciati quando il cluster cambia dimensione. *Consistent hashing.*

Tempo e rilevazione

- ▶ **Ordinamento degli eventi** — stabilire se un evento è accaduto prima di un altro in assenza di un clock condiviso. *Logical clock (Lamport), vector clock.*
- ▶ **Snapshot globale** — fotografare lo stato dell'intero sistema in modo coerente senza fermare i nodi. *Chandy-Lamport.*
- ▶ **Failure detection** — rilevare quali nodi sono crashati con le garanzie del modello scelto. *Timeout adattivo, $\diamond P$.*
- ▶ **Membership** — tenere traccia di quali nodi fanno parte del cluster in ogni momento. *SWIM, Zookeeper.*